Create a JSON object in javascript and print the contents of the object

Ex:1 JSON.parse()

json converting to object

hello

Ex:1 JSON.parse()

```
<html>
 <body>
 <h1>json converting to object</h1>
 <p id="demo"> hello</p>
 <script>
 const txt='{"name":"Shreshta", "age":19, "place":"Sakleshpur"}';
 const myobj=JSON.parse(txt);
 document.getElementById("demo").innerHTML=myobj.name;
 </script>
 </body>
</html>
```

Ex2: JSON.stringify()

```
<html>
 <body>
 <h1>json converting to string using stringify</h1>
 <p id="demo"> Hello</p>
 <script>
 const person={name:"Shreshta", age:19, place:"sakleshpur"}
 const txt=JSON.stringify(person);
 document.getElementById("demo").innerHTML=txt;
 </script>
 </body>
</html>
```

Write a program to demonstrate props and state in react.

```
import React from 'react';
import ReactDOM from 'react-dom/client';
function Car(props) {
 return <h2>I am a { props.brand }!</h2>;
}
function Garage() {
 return (
 <>
 <h1>Who lives in my garage?</h1>
 <Car brand="Ford" />
 </>
 );
}
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Garage />);
```

Build a college website using react components.

Header.js

```
function Header()
{
return (
<>
<table bgcolor=”yellow” border="0" width="100%">
<tr>
<th>HOME</th>
```

```
<th>ADMISSION</th>
<th>DEPARTMENT</th>
<th>CONTACTS</th>
</tr>
</table>
<hr color=”red”/>
</>
)
}
export default Header;
```

Section.js

```
function Section()
{
return (
<>
<h1 style={{color:"green"}} align="center">WELCOME TO SDM
POLYTECHNIC,</h1><br/>
<h2 style={{color:"green"}} align="center">UJIRE, 574240</h2>
<table align="center" borde="0" width="100">
<tr>
<td>
<img src="sdmpimg.jpg" height="400" width="350" alt="clg img"/>
</td>
<td>
<h3><i>Information about SDMP:</i></h3>
<pre>
```

SDM Polytechnic Ujire, established in 2008, is an institution managed by the SDM Educational Society, offering diploma courses in Civil, Computer Science, Electronics & Communication, and Mechanical Engineering.
The college is located in Ujire, Karnataka, at the foothills of the Western Ghats.
It's part of the larger SDM Educational Society, which runs 56 educational institutions.
The college is recognized for its unique initiatives in cultural and sports activities and is known for its early implementation of the National Education Policy (NEP).
Key Aspects of SDM Polytechnic Ujire:
Management: Managed by the SDM Educational Society, Ujire.
Diploma Courses: Offers 3-year diploma courses in Civil, Computer Science, Electronics & Communication, and Mechanical Engineering.
Location: Situated in Ujire, at the foothills of the Western Ghats.
Unique Initiatives: Known for cultural and sports initiatives, and early implementation of the National Education Policy (NEP).
Research: Recognized center for research programs affiliated with Mangalore University,
Tumkur University, and Kannada University Hampi.
Autonomous Status: SDM College, Ujire, is an autonomous college under Mangalore University,
according to Wikipedia.
Library: The library is automated with an in-house developed software and offers a wide range
of books for students.
Departments: SDM Polytechnic has departments for Civil, Computer Science, Electronics &
Communication, and Mechanical Engineering.
Campus Life: Provides facilities like a library, and various extracurricular activities.

); } export default Section;

Footer.js

```
function Footer()

{

return (

<>

<hr color=”red”/>

<h3><u>Image Gallery</u></h3>
```

```
<table width="100%">
<tr>
<td><img src="sdmp1.jpg" width="450" heigth="150"/></td>
<td><img src="sdmp2.jpg" width="450" heigth="150"/></td>
<td><img src="sdmp3.jpg" width="450" heigth="150"/></td>
</tr>
<tr>
<td><img src="sdmp4.jpg" width="450" heigth="150"/></td>
<td><img src="sdmp5.jpg" width="450" heigth="150"/></td>
<td><img src="sdmp6.jpg" width="450" heigth="150"/></td>
</tr>
</table>
<hr color=”red”/>
<table border="0" width="1000">
<tr>
<th align="center">Contacts</th>
<th align="center">Quick links</th>
<th align="center">Also Explore</th>
</tr>
<tr>
<td align="center">phone: 082562 36600</td>
<td align="center">Student Space</td>
<td align="center">Collage magzine</td>
</tr>
<tr>
<td align="center">SDM Education Society</td>
<td align="center">About Us</td>
<td align="center">Gallery</td>
```

```
</tr>
<tr>
<td align="center">Ujire-574240</td>
<td align="center">Alumni</td>
<td align="center">Video clip</td>
</tr>
</table>
copywrite@shreshtayr
</>
);
}
export default Footer;
```

index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import Header from './Header';
import Section from './Section';
import Footer from './Footer';
const root=ReactDOM.createRoot(document.getElementById("root"));
root.render(<> <Header /> <Section /> <Footer /> </>);
```

Create a convert to upper case and convert to lower case buttons

Textconvertor.js

```
import React,{useState} from
'react'; function TextConvertor(){
const [text,setText]=useState("");
```

```
const handleUppercase = () =>{
setText(text.toUpperCase());
}
const handleLowercase = () =>{
setText(text.toLowerCase());
}
return(
<div>
<center>
<textarea value = {text} onChange={(e)=>setText(e.target.value)}/><br/><br/>
<button onClick = {handleUppercase}>Convert to
Uppercase</button>   <button onClick =
{handleLowercase}>Convert to Lowercase</button> </center>
</div>
);
}
export default TextConvertor
```

Index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import TextConvertor from
'./textconvertor'; const result = (
<TextConvertor/>
);
const root =
ReactDOM.createRoot(document.getElementById('root'));
root.render(result);
```

Explain React Hooks with an example
1] UseState Hook:

App2.js

```
import React ,{useState}from 'react' ;
export default function App2()
{
const[color,setcolor]=useState("Red"); return (
<div><h1>My favorite color={color}</h1>
<button onClick={()=>setcolor("Blue")}>Change Color</button> </div>
);
}
```

Index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App2 from './App2';
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App2/>);
```

2]UseEffect Hook:

App3.js

```
import React, { useState, useEffect } from 'react';
export default function App3() {
const [count, setCount] = useState(0); useEffect(() => {
const timer = setTimeout(() => { setCount((prevCount) => prevCount + 1);
}, 1000);
return () => clearTimeout(timer);
```

```
}, [count]); return (<div>
<h1>Counter value = {count}</h1>
<h2>I have rendered {count} times</h2>
</div>
);
}
```

Index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App3 from './App3';
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App3/>);
```

3]UseContext Hook

App4.js

```
import React, { useContext, createContext, useState } from 'react';
const UserContext = createContext();
export default function Component1() {
const [user, setUser] = useState("Shreshta");
return (<>
<UserContext.Provider value={user}>
<h1>Hello {user}</h1>
<h2>Component 1</h2>
<Component2 />
</UserContext.Provider>
</> ); }
function Component2() {
const user = useContext(UserContext);
```

```
return (
<div>
<h1>Hello {user}</h1>
<h2>Component 2</h2>
<Component3 />
</div>
); }
function Component3() {
const user = useContext(UserContext);
return ( <div>
<h1>Hello {user}</h1>
<h2>Component 3</h2>
</div>
); }
```

Index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import Component1 from './App4';
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render (<Component1/>);
```

4]UseRef Hook

App5.js

```
import React, { useRef } from 'react';
function Textfocus() {
const inputRef = useRef(null);
const handleFocus = () => {
```

```
inputRef.current.focus();
};
return (
<div>
<h1>useRef Example</h1>
<input ref={inputRef} type="text" />
<button onClick={handleFocus}>click to focus on Input</button>
</div> ); }
export default Textfocus;
```

index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import Textfocus from './Lab/App5';
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Textfocus/>);
```

React router example

Index.js

```
import {createRoot} from 'react-dom/client';
import { BrowserRouter, Routes,Route,Link } from 'react-router-dom';
function Home()
{
return <h1> Home page</h1>;
}
function About()
```

```
{
return <h1> About page</h1>;
}
function Contact()
{
return<h1> Contact page</h1>;
}
function App()
{
return (
<BrowserRouter>
<nav>
<Link to='/'>Home</Link>|{""}
<Link to='/About'>ABOUT</Link>|{""}
<Link to='/Contact'>CONTACT</Link>
</nav>
<Routes>
<Route path="/" element={<Home/>}/>
<Route path="/About" element={<About/>}/>
<Route path="/Contact" element={<Contact/>}/>
</Routes>
</BrowserRouter>
);
}
createRoot(document.getElementById("root")).render(<App/>);
```

Illustrate setter and constructor DI's in Spring Boot

Setter DI

DemoAppApplication.java

```
package com.example.DemoApp;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
@SpringBootApplication
public class DemoAppApplication {
public static void main(String[] args)
{
ApplicationContext context =
SpringApplication.run(DemoAppApplication.class, args);
 Student student=context.getBean(Student.class);
 student.Greet();
}
}
```

Student.java

```
package com.example.DemoApp;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
@Component
public class Student {
 private Teacher teacher;
 @Autowired
 public void setTeacher(Teacher teacher)
 {
 this.teacher=teacher;
 }
 public void Greet()
 {
 teacher.MSG();
 System.out.println("hello from student class");
 }
}
```

Teacher.java

```
package com.example.DemoApp;
import org.springframework.stereotype.Component;
@Component
```

```
public class Teacher {
 public void MSG(){
 System.out.println("Hello from Teacher class");
 }
}
```

Constructor DI

DemoAppApplication.java

```
package com.example.DemoApp;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
@SpringBootApplication
public class DemoAppApplication {
public static void main(String[] args)
 {
ApplicationContext context = SpringApplication.run(DemoAppApplication.class,
args);
Student student=context.getBean(Student.class);
student.Greet();
}
}
```

Student.java

```
package com.example.DemoApp;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
@Component
public class Student {
 public Teacher teacher;
 public Student(Teacher teacher)
 {
this.teacher=teacher;
 }
 public void Greet()
 {
 teacher.MSG();
 System.out.println("hello from student class");
 }
}
```

Teacher.java

```
package com.example.DemoApp;
```

```
import org.springframework.stereotype.Component;
@Component
public class Teacher {
 public void MSG(){
 System.out.println("Hello from Teacher class");
 }
}
```

Demonstrate CRUD operations using H2 database.

Student.java

```
package com.example.RestExample.Model;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import org.springframework.stereotype.Component;
@Component
@Entity
public class Student {
 @Id
 private int roll;
 private String name;
 private int marks;
 public int getRoll() {
 return roll;
 }
 public void setRoll(int roll) {
 this.roll = roll;
 }
 public String getName() {
 return name;
 }
 public void setName(String name) {
 this.name = name;
 }
 public int getMarks() {
 return marks;
 }
 public void setMarks(int marks) {
 this.marks = marks;
 }
 publicthis.roll = roll;
```

```
 this.name = name;
 this.marks = marks;
 }
 public Student() {
 }
}
```

StudentController.java

```
package com.example.RestExample.Controller;
import com.example.RestExample.Model.Student;
import com.example.RestExample.Service.StudentService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
public class StudentController {
 @Autowired
 StudentService service;
 @GetMapping("/students")
 public List<Student> getAllStudentDetails(){
 return service.getAllStudents();
 }
 @GetMapping("/students/{stid}")
 public Student getStudentById(@PathVariable int stid){
 return service.getStudentById(stid);
 }
 @PostMapping("/students")
 public void addStudentDetails(@RequestBody Student s){
 service.addStudent(s);
 }
 @PutMapping("/students")
 public void updateStudents(@RequestBody Student stud){
 service.updateStudent(stud);
 }
 @DeleteMapping("students/{stid}") Student(int roll, String name, int marks) {
public void deleteStudentById(@PathVariable int stid){
 service.deleteStudentById(stid);
 }
}
```

StudentService.java

```
package com.example.RestExample.Service;
```

```
import com.example.RestExample.Model.Student;
import com.example.RestExample.Repository.StudentRepo;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
@Service
public class StudentService {
 @Autowired
 StudentRepo repo;
 /*List<Student> students= new ArrayList<>(
 Arrays.asList(new Student(111,"Kumar",45),
 new Student(112,"Sathish",80),
 new Student(113,"Roshan",95)));*/
 public List<Student> getAllStudents(){
 return repo.findAll();
 }
 public Student getStudentById(int stid){
 return repo.findById(stid).orElse(new Student());
 }
 public void addStudent(Student s){
 repo.save(s);
 }
 public void updateStudent(Student stud) {
 repo.save(stud);
 }
public void deleteStudentById(int stid) {
 repo.deleteById(stid);
 }
}
```

StudentRepo.java

```
package com.example.RestExample.Repository;
import com.example.RestExample.Model.Student;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
@Repository
public interface StudentRepo extends JpaRepository<Student,Integer> {
}
```

application.properties

```
spring.application.name=RestExample
spring.datasource.username=varada
spring.datasource.password=varada
spring.datasource.url=jdbc:h2:mem:sdmp
spring.datasource.driverClassName=org.h2.Driver
spring.h2.console.enabled=true
spring.jpa.show-sql=true
```

Demonstrate CRUD operations using MySQL.

application.properties

```
spring.application.name=SpringMySQLDemo
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=Mysql
spring.jpa.generate-ddl=true
spring.jpa.hibernate.ddl.auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
spring.jpa.show-sql=true
```

Student.java

```
import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
@Entity
@Table(name="Student_info")
public class Student {
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name="stid")
private int id;
@Column(name="stname")
private String name;
@Column(name="stmarks")
private int marks;
public int getId() {
```

```
return id;
}
public void setId(int id) {
this.id = id;
}
public String getName() {
return name;
}
public void setName(String name) {
this.name = name;
}
public int getMarks() {
return marks;
}
public void setMarks(int marks) {
this.marks = marks;
}
public Student(int id, String name, int marks) {
this.id = id;
this.name = name;
this.marks = marks;
}
public Student() {
}
}
```

StudentController.java

```
package com.example.SpringMySQLDemo.Controller;
import com.example.SpringMySQLDemo.Model.Student;
import com.example.SpringMySQLDemo.Service.StudentService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
public class StudentController {
@Autowired
StudentService service;
```

```
@PostMapping("/students")
public void postDetails(@RequestBody Student s){
service.addStudent(s);
}
```



```
@GetMapping("/students")
public List<Student> getAllData(){
return service.getAllData();
}
@GetMapping("/students/{stid}")
public Student getStudentById(@PathVariable int stid){
return service.getStudentByid(stid);
}
@DeleteMapping("/students/{stid}")
public void deleteById(@PathVariable int stid){
service.deleteById(stid);
}
@PutMapping("/students")
public void updateDetails(@RequestBody Student s){
service.updateStudent(s);
}
}
```

StudentService.java

```
package com.example.SpringMySQLDemo.Service;
import com.example.SpringMySQLDemo.Model.Student;
import com.example.SpringMySQLDemo.Repository.StudentRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;
@Service
public class StudentService {
@Autowired
StudentRepository repo;
public String addStudent(Student s) {
```

```
repo.save(s);
return "stored in database";
}
public List<Student> getAllData() {
return repo.findAll();
}
public void deleteById(int stid) {
repo.deleteById(stid);
}
public void updateStudent(Student s) {
repo.save(s);
}
public Student getStudentByid(int stid) {
return repo.findById(stid).orElse(new Student());
}
}
```

StudentRepository.java

```
package com.example.SpringMySQLDemo.Repository;
import com.example.SpringMySQLDemo.Model.Student;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
@Repository
public interface StudentRepository extends JpaRepository<Student,Integer> {
}
```

Insert records into the books collection with suitable fields.

```
>>use library
db.books.insertMany([
{
title: ”The great adventure”,
author: ”Jhon Adams”,
price:299,
discountPrice:249,
publisher: ”Adventure books”,
tags: [“fiction”, “adventure”, “bestseller”],
reviews: [
{reviewer:”Alice”, rating:4, comment:”Excellent book!”},
{reviewer:”Bob”, rating:3, comment:”every enjoyable”}
],
publishedDate:”2021-05-15”,
available: true
},
{
title: ”Learning MongoDB”,
```

author: ”Jane Smith”,
price:399,
discountPrice:359,
publisher: ”Tech Press”,
tags: [“non-fiction”, “technology”, “database”],
reviews: [
{reviewer:”Charli”, rating:5, comment:”Very informative”}
],
publishedDate:”2022-08-20”,

available: false
},
{
title: ”Cooking with love”,
author: ”James Joyee”,
price:150.50,
tags: [“non-fiction”, “cooking”, “bestseller”],
reviews: [
{reviewer:”Fiona”, rating:3, comment:”Great recipes”},
{reviewer:”David”, rating:4, comment:”Good”},
{reviewer:”George”, rating:3, comment:”loved the dessert section”}
],
publishedDate:”2020-11-30”,
available: true
}])

Find all books tagged with both non-fiction and lifestyle

- db.books.find({tags:{$all:["non-fiction", "lifestyle"]}})

Find all books that have a review with a rating of 5 and a comment containing "great"

‘great’.

- db.books.find({reviews: {$elemMatch: {rating: { $gte: 3 },comment: {

$regex:"great", $options:"i" }}}})

Find all books where the name of the publisher ends with ‘press’.

- db.books.find({publisher: {$regex:/press$/,$options: "i"}})

output:

Find books where the word MongoDB appears in the title field only.

- db.books. createIndex({title:"text"})
- db.books.find({$text:{$search:"MongoDB"}})

Retrieve all books sorted by discounted price in descending order.

- db.books.find().sort({discountedPrice:-1})

Update the price of Learning MongoDB to 450.

- db.books. updateOne({title:"Learning MongoDB"}, {$set:{price:450}})

Count books by author.

- db.books. .aggregate([{$group:{_id:"$author", totalBook:{$sum:1}}}])

33. Find the maximum price of books.

 db.books. aggregate([{$group:{_id:"null", minDiscountedPrice:

{$min:"$price"}}}])

35. Remove the books titled cooking with love from the collection.

 db.books.deleteOne()

CRUD Operations- Springboot with MongoDB

Student.java

```
package com.example.Studentdemo.model;
import org.springframework.data.mongodb.core.mapping.Document;
@Document (collection = "StudentDB")
public class Student {
private int id;
private String name;
private String regNo;
private int sem;
public Student() {
}
public Student(int id, String name, String regNo, int sem) {
this.id = id;
this.name = name;
this.regNo = regNo;
this.sem = sem;
}
public int getId() {
return id;
}
public void setId(int id) {
this.id = id;
}
public String getName() {
return name;
}
public void setName(String name) {
this.name = name;
}
```

```
public String getRegNo() {
return regNo;
}
public void setRegNo(String regNo) {
this.regNo = regNo;
}
public int getSem() {
return sem;
}
public void setSem(int sem) {
this.sem = sem;
}
}
```

StudentRepository.java

```
package com.example.Studentdemo.repository;
import com.example.Studentdemo.model.Student;
import org.springframework.data.mongodb.repository.MongoRepository;
public interface StudentRepository extends
MongoRepository <Student, Integer>
{
}
```

StudentController.java

```
package com.example.Studentdemo.controller;
import com.example.Studentdemo.model.Student;
import com.example.Studentdemo.repository.StudentRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
@RequestMapping("/student")
public class StudentController {
@Autowired
private StudentRepository studentRepository;
@PostMapping("/post")
public String addData(@RequestBody Student student)
{
```

```
try{
this.studentRepository.save(student);
return "Data inserted successfully";
} catch (Exception e) {
return "Data not inserted";
}
}
@GetMapping("/data")
public List<Student> getData()
{
return (List<Student>) this.studentRepository.findAll();
}
@DeleteMapping("/delete")
public String deleteAllData()
{
try{
this.studentRepository.deleteAll();
return "data deleted successfully";
} catch (Exception e) {
return "data not deleted";
}
}
@DeleteMapping("/{id}")
public String deleteAllData(@PathVariable int id)
{
try{
this.studentRepository.deleteById(id);
return "Data deleted successfully";
} catch (Exception e) {
return "Data not deleted";
}
}
@PutMapping("/{id}")
public String updateData(@RequestBody Student student, @PathVariable int id)
{
try{
this.studentRepository.save(student);
```

```
return "Data updated successfully";
} catch (Exception e) {
return "Data not updated";
}
}
}
```

application.properties

```
spring.data.mongodb.host=localhost
spring.data.mongodb.port=27017
spring.data.mongodb.database=myDB
```

StudentdemoApplication.java

```
package com.example.Studentdemo;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class StudentdemoApplication {

public static void main(String[] args) {
SpringApplication.run(StudentdemoApplication.class, args);
}
}
```

CREATE DOCKER CONTAINER FROM DOCKER IMAGE AND RUN

```
from flask import Flask
app = Flask(__name__)
@app.route("/")
def home():
return "Thank you"
if __name__=='__main__':
app.run(debug=True)
```

Dockerfile

```
FROM python:3.8-slim
WORKDIR /app
COPY . /app
RUN pip install --no-cache-dir -r requirement.txt
EXPOSE 5000
ENV FLASK_APP=app.py
```

CMD ["flask","run","--host=0.0.0.0"]

requirement.txt

Flask==2.3.2

will build the image: “docker build -t nano .“

run the file inside Docker desktop =”docker run -p 5000:5000 nano”